01 – Internet & HTTP

Request and Response

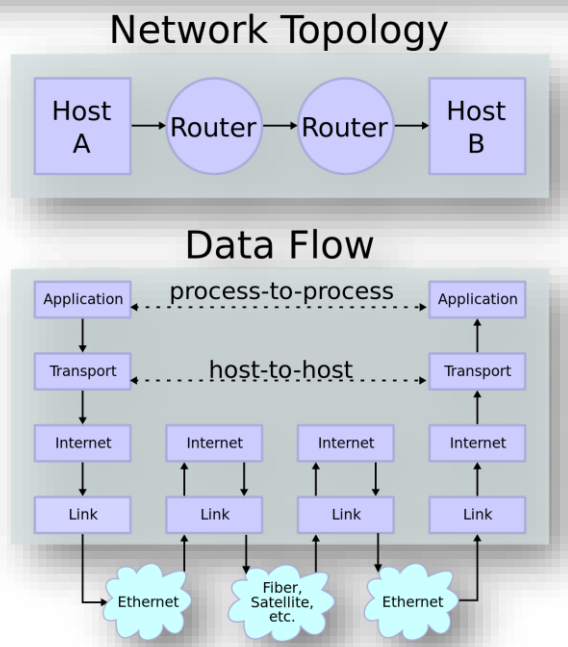
Jérôme Landré – **jerome.landre@ju.se**

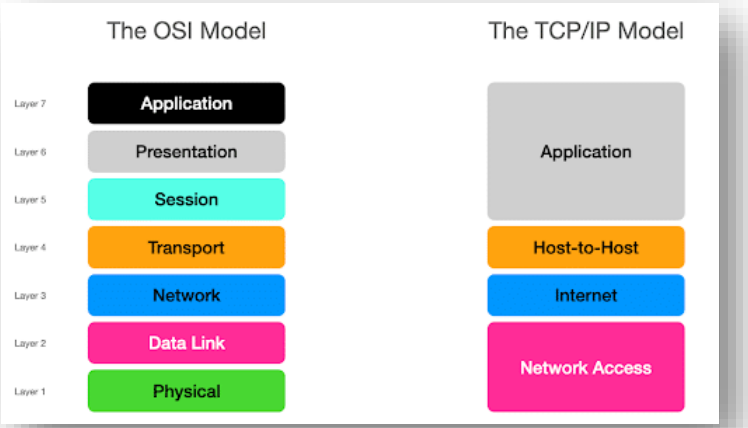
OUTLINE

1) Internet
2) HTTP
3) Request and Response

1) INTERNET

- **Internet** is a network of networks
- It is based on the **TCP/IP** protocols
- It has a **4-layer** model:
  - **Application**
  - **Transport (TCP & UDP)**
  - **Internet (IP)**
  - **LINK (network access)**





Images: https://brightlineit.com/wp-content/uploads/2017/06/170613-upgrade-your-internet-connection.jpg,
https://images.tmcnet.com/tmc/misc/articles/image/2020-sep/AdobeStock_271380596-network-global-connection-connectivity-interconnect-supersize.jpeg,
https://en.wikipedia.org/wiki/Internet_protocol_suite#/media/File:IP_stack_connections.svg,
https://netbeez.net/wp-content/uploads/2022/02/tcp-ip-model.png

1) INTERNET

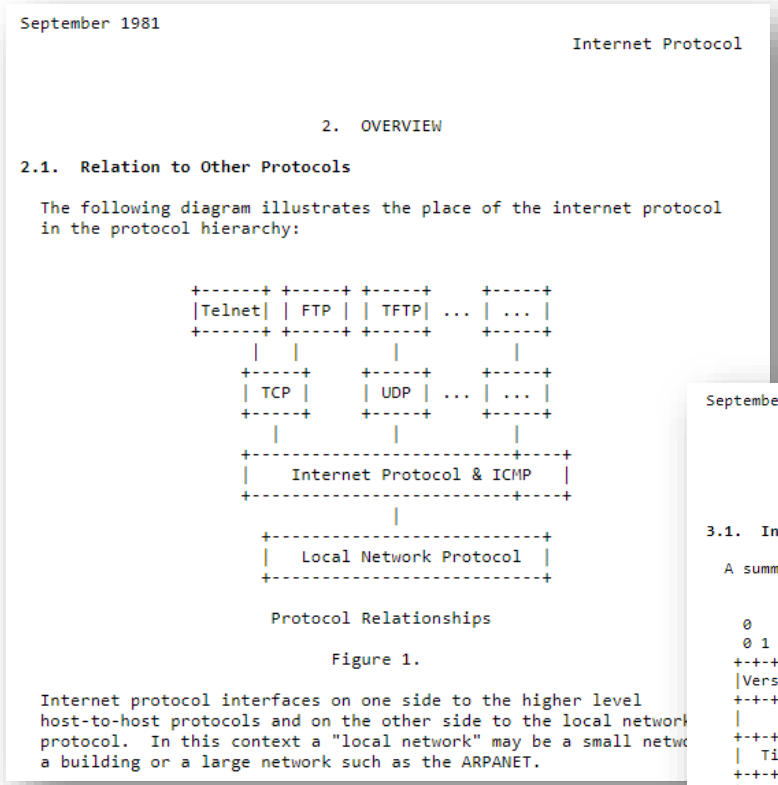
- **Internet** is a network of networks
- It is based on the **TCP/IP** protocols
- It has a **4-layer** model:
  - **Application**
  - **Transport (TCP & UDP)**
  - **Internet (IP)**
  - **LINK (network access)**

1) INTERNET

- **Internet** is based on **RFC (Request For Comments)** open/free documents

  https://www.rfc-editor.org/

- Internet doesn't belong to anyone so it belongs to everyone



September 1981 Internet Protocol

2. OVERVIEW

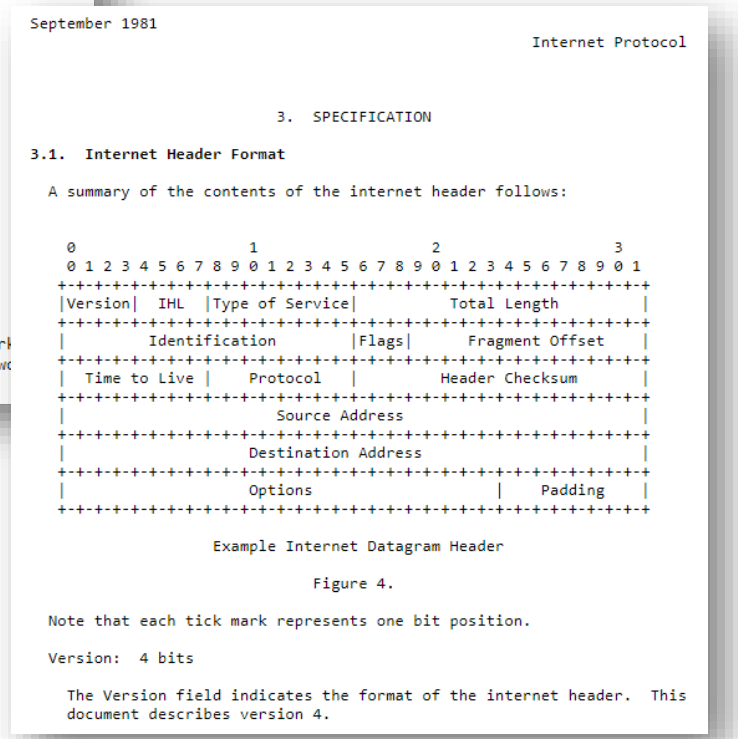
2.1. Relation to Other Protocols

The following diagram illustrates the place of the internet protocol in the protocol hierarchy:

Protocol Relationships

Figure 1.

Internet protocol interfaces on one side to the higher level host-to-host protocols and on the other side to the local network protocol. In this context a "local network" may be a small netwo a building or a large network such as the ARPANET.



September 1981 Internet Protocol

3. SPECIFICATION

3.1. Internet Header Format

A summary of the contents of the internet header follows:

Example Internet Datagram Header

Figure 4.

Note that each tick mark represents one bit position.

Version: 4 bits

The Version field indicates the format of the internet header. This document describes version 4.

RFC791: Internet Protocol

1) INTERNET

- **Internet** is based on **RFC (Request For Comments)** open/free do

September 1981
Internet Protocol

2. OVERVIEW

2.1. Relation to Other Protocols

The following diagram illustrates the place of the internet protocol in the protocol hierarchy:

+------+ +-----+ +-----+ +-----+
|Telnet| | FTP | | TFTP| ... | ... |
+------+ +-----+ +-----+ +-----+

Internet Datagram Header

Figure 4.

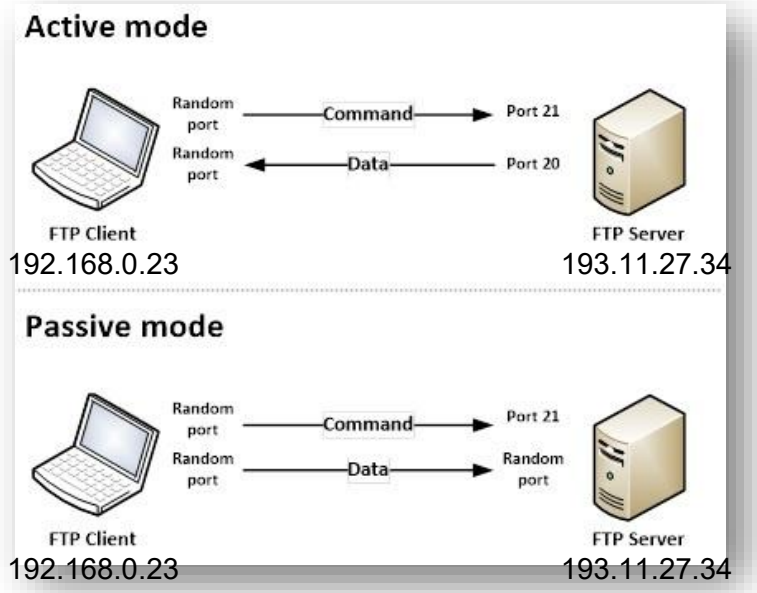
Note that each tick mark represents one bit position.

Version: 4 bits

The Version field indicates the format of the internet header. This document describes version 4.

1) INTERNET

- Every computer has an **IP address** to exchange data with other computers
- In IPv4, addresses are four bytes long:
  - A.B.C.D with A, B, C, D = {0, …, 255} in decimal form
- In IPv6, addresses are sixteen bytes long:
  - M:N:O:P:Q:R:S:T with M, N, …, O, P = {0, …, 0xffff} in hexadecimal form
- Every IP address has possibly **65536 ports** to define **network services**
- An IP and a port is a **network socket**



Images: https://www.to-increase.com/hs-fs/hubfs/Imported_Blog_Media/cs-ftp-1-1.jpg?width=430&height=339&name=cs-ftp-1-1.jpg

1) INTERNET

- Just a small remark on **IP addresses:**
  - In general, **address finishing by 0s (one or more)** are **network addresses** (referencing a **whole set of IP addresses**)
  - In general, **address finishing by 255s (one or more) are broadcast addresses** (used to send **to all the machines** on the IP network)
  - But it depends of the **network mask** so it is not **always true**

  - If you are interested, you can read more there:

1) INTERNET

- Among these **65536 ports**:
  - 0 to 1023 are **Well-Known Ports**:
    - **20, 21**: File Transfer Protocol (FTP) for data and control
    - **22**: Secure Shell (SSH) for secure logins, file transfers, and port forwarding
    - **25**: Simple Mail Transfer Protocol (SMTP) for email routing
    - **53**: Domain Name System (DNS) for translating domain names to IP addresses
    - **80**: Hypertext Transfer Protocol (HTTP) for web traffic
    - **443**: HTTP Secure (HTTPS) for secure web traffic

Port ranges

Start	End	Designation
0	1023	System or well-known ports
1024	49151	User or registered ports
49152	65535	Dynamic, private or ephemeral ports

Images: https://en.wikipedia.org/wiki/Registered_port

1) INTERNET

- Among these **65536 ports**:
  - 1024 to 49151 are **Registered Ports**:
    - **1234:** VLC Media Player
    - **1433:** MSSQL database server
    - **3306**: MySQL database server
    - **3389**: Microsoft Terminal Server (RDP)
    - **3478-3481**: Microsoft Teams
    - **6463-6472**: Discord
    - **8080**: Alternative port for http
  - 49152 to 65535 are **Dynamic Ports**

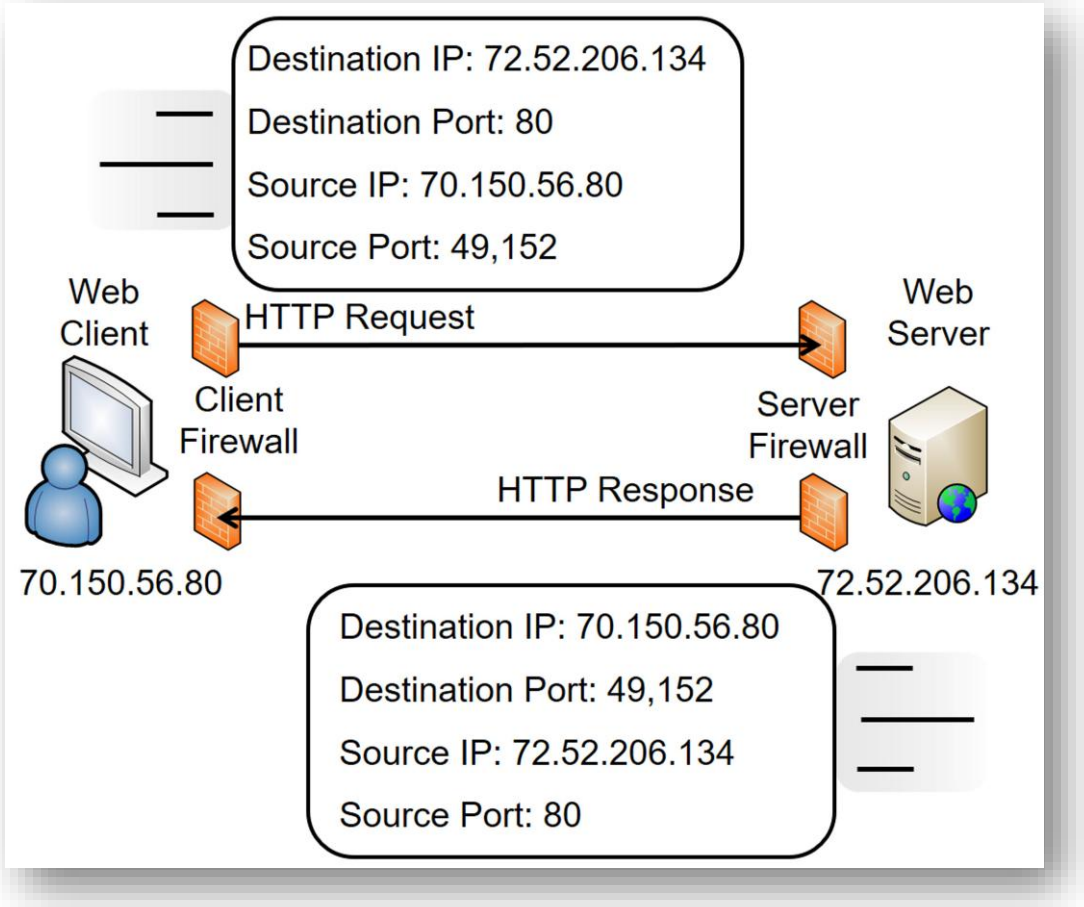
Port ranges

Start	End	Designation
0	1023	System or well-known ports
1024	49151	User or registered ports
49152	65535	Dynamic, private or ephemeral ports

Images: https://en.wikipedia.org/wiki/Registered_port

2) HTTP

- HyperText Transfer Protocol is the protocol used to **exchange information** between a **client** and a **server**
- The information is mainly **text**
- The client sends a **request** (in text format) and gets back a **response** from the server
- The **port 80** is used by the client to connect to an http server

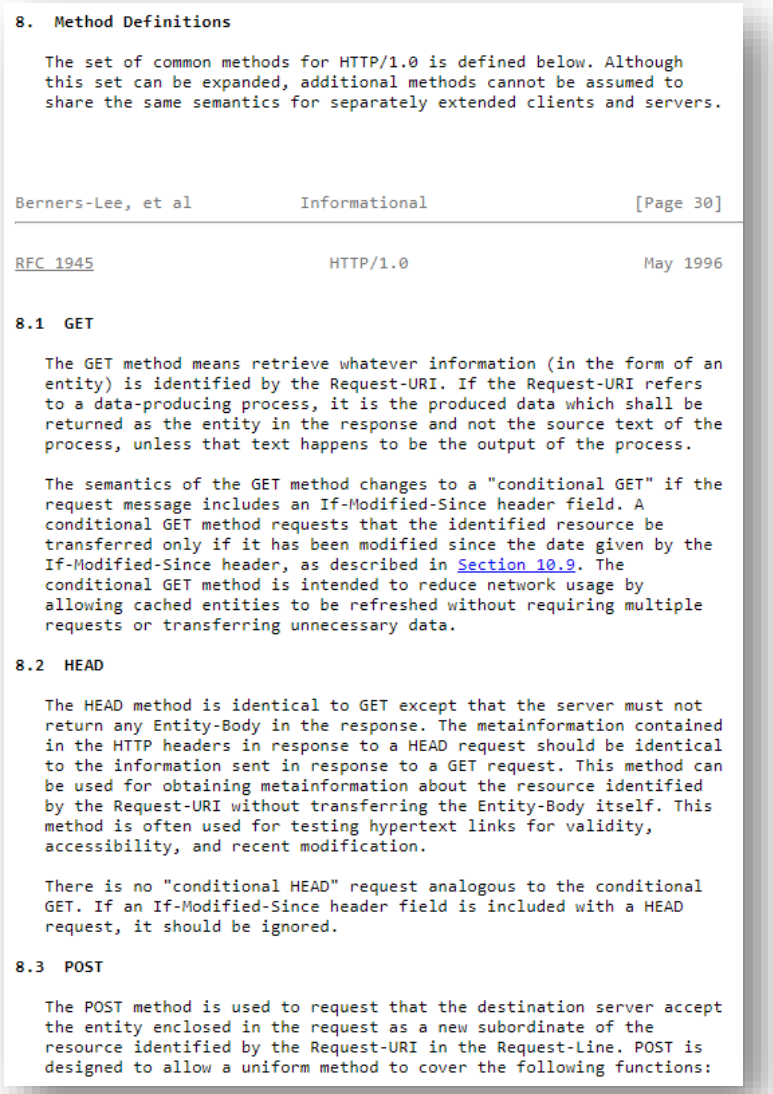


Images: https://blogs.getcertifiedgetahead.com/wp-content/uploads/2019/12/Using-Source-and-Destination-Ports-610x508.jpg

2) HTTP

- **HTTP/1.0 (RFC1945)**

- RFC official website:
  - https://www.rfc-editor.org/rfc-index.html



8. Method Definitions

The set of common methods for HTTP/1.0 is defined below. Although this set can be expanded, additional methods cannot be assumed to share the same semantics for separately extended clients and servers.

Berners-Lee, et al Informational [Page 30]

RFC 1945 HTTP/1.0 May 1996

8.1 GET

The GET method means retrieve whatever information (in the form of an entity) is identified by the Request-URI. If the Request-URI refers to a data-producing process, it is the produced data which shall be returned as the entity in the response and not the source text of the process, unless that text happens to be the output of the process.

The semantics of the GET method changes to a "conditional GET" if the request message includes an If-Modified-Since header field. A conditional GET method requests that the identified resource be transferred only if it has been modified since the date given by the If-Modified-Since header, as described in Section 10.9. The conditional GET method is intended to reduce network usage by allowing cached entities to be refreshed without requiring multiple requests or transferring unnecessary data.

8.2 HEAD

The HEAD method is identical to GET except that the server must not return any Entity-Body in the response. The metainformation contained in the HTTP headers in response to a HEAD request should be identical to the information sent in response to a GET request. This method can be used for obtaining metainformation about the resource identified by the Request-URI without transferring the Entity-Body itself. This method is often used for testing hypertext links for validity, accessibility, and recent modification.

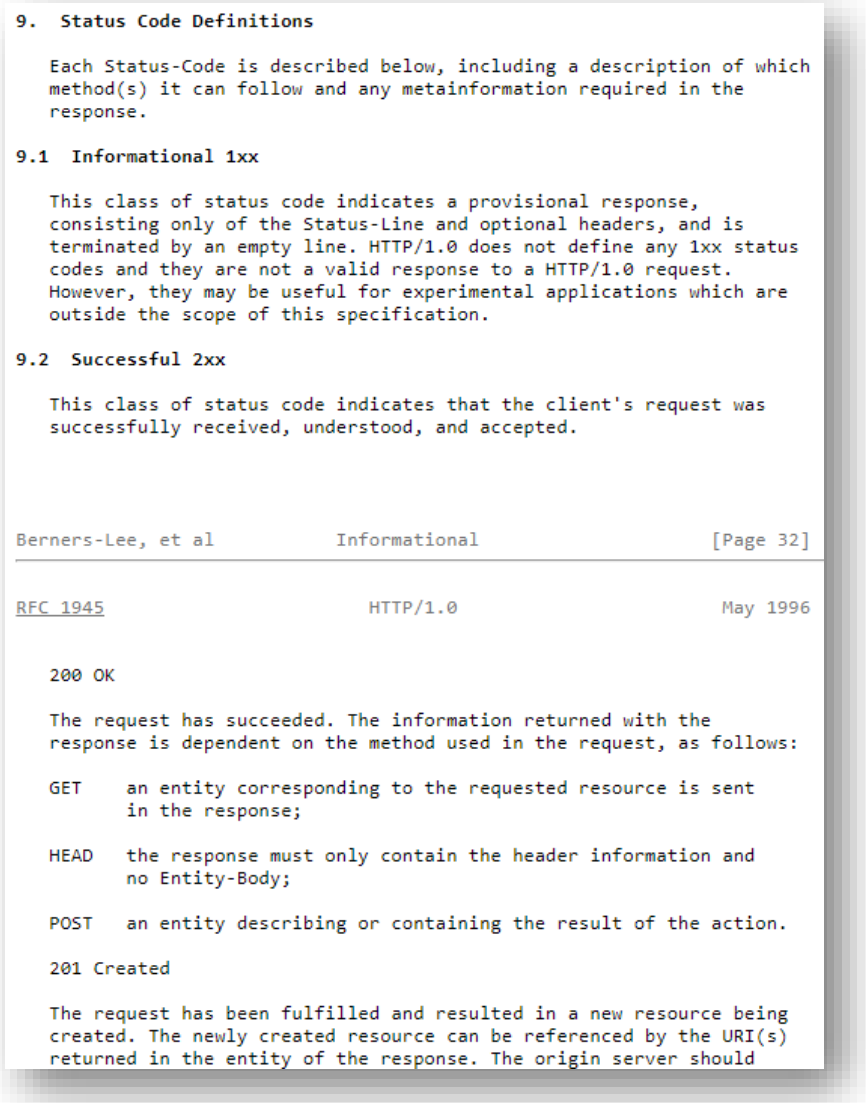
There is no "conditional HEAD" request analogous to the conditional GET. If an If-Modified-Since header field is included with a HEAD request, it should be ignored.

8.3 POST

The POST method is used to request that the destination server accept the entity enclosed in the request as a new subordinate of the resource identified by the Request-URI in the Request-Line. POST is designed to allow a uniform method to cover the following functions:

2) HTTP

- **HTTP/1.0 (RFC1945)**
  - **Obsolete!**



9. Status Code Definitions

Each Status-Code is described below, including a description of which method(s) it can follow and any metainformation required in the response.

9.1 Informational 1xx

This class of status code indicates a provisional response, consisting only of the Status-Line and optional headers, and is terminated by an empty line. HTTP/1.0 does not define any 1xx status codes and they are not a valid response to a HTTP/1.0 request. However, they may be useful for experimental applications which are outside the scope of this specification.

9.2 Successful 2xx

This class of status code indicates that the client's request was successfully received, understood, and accepted.

Berners-Lee, et al Informational [Page 32]

RFC 1945 HTTP/1.0 May 1996

200 OK

The request has succeeded. The information returned with the response is dependent on the method used in the request, as follows:

GET an entity corresponding to the requested resource is sent in the response;

HEAD the response must only contain the header information and no Entity-Body;

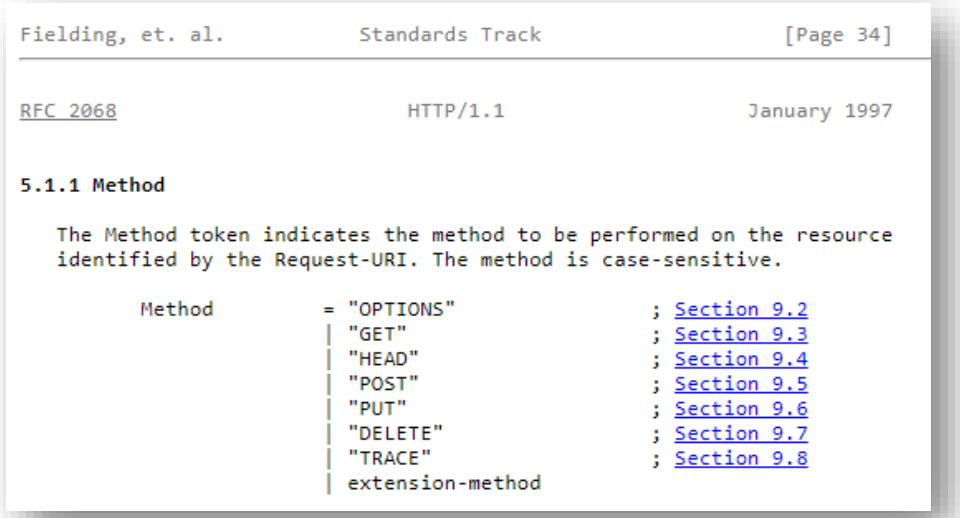
POST an entity describing or containing the result of the action.

201 Created

The request has been fulfilled and resulted in a new resource being created. The newly created resource can be referenced by the URI(s) returned in the entity of the response. The origin server should

2) HTTP

- **HTTP/1.1 (RFC2068)**
  - **Obsolete!**
- **HTTP/1.1 (RFC2616)**
  - **Obsolete!**



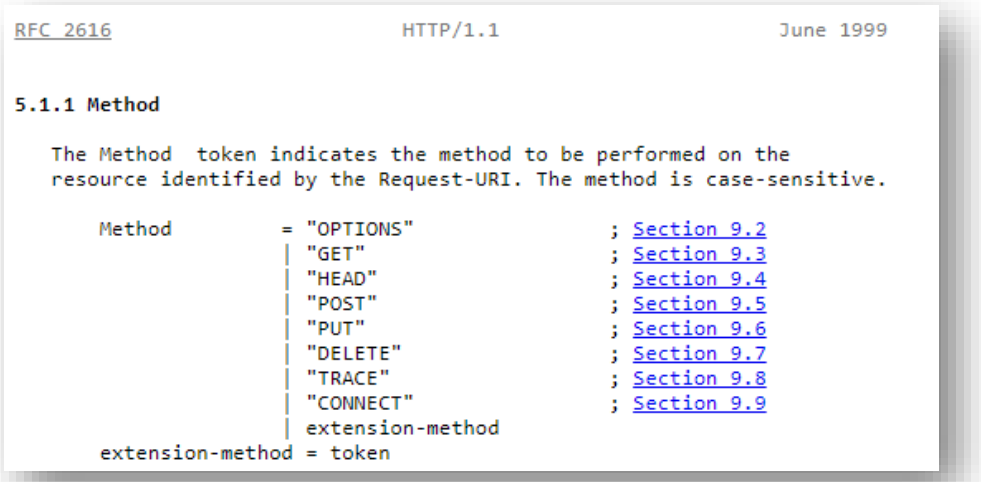
Fielding, et. al. Standards Track [Page 34]

RFC 2068 HTTP/1.1 January 1997

5.1.1 Method

The Method token indicates the method to be performed on the resource identified by the Request-URI. The method is case-sensitive.

```
Method         = "OPTIONS"                ; Section 9.2
               | "GET"                    ; Section 9.3
               | "HEAD"                   ; Section 9.4
               | "POST"                   ; Section 9.5
               | "PUT"                    ; Section 9.6
               | "DELETE"                 ; Section 9.7
               | "TRACE"                  ; Section 9.8
               | extension-method
```



RFC 2616 HTTP/1.1 June 1999

5.1.1 Method

The Method token indicates the method to be performed on the resource identified by the Request-URI. The method is case-sensitive.

```
Method         = "OPTIONS"                ; Section 9.2
               | "GET"                    ; Section 9.3
               | "HEAD"                   ; Section 9.4
               | "POST"                   ; Section 9.5
               | "PUT"                    ; Section 9.6
               | "DELETE"                 ; Section 9.7
               | "TRACE"                  ; Section 9.8
               | "CONNECT"                ; Section 9.9
               | extension-method
extension-method = token
```

2) HTTP

- **HTTP/1.1 (RFC7230, …, RFC7235)**
  - **Obsolete!**

2.1. Client/Server Messaging

HTTP is a stateless request/response protocol that operates by exchanging messages (Section 3) across a reliable transport- or session-layer "connection" (Section 6). An HTTP "client" is a program that establishes a connection to a server for the purpose of sending one or more HTTP requests. An HTTP "server" is a program that accepts connections in order to service HTTP requests by sending HTTP responses.

The terms "client" and "server" refer only to the roles that these programs perform for a particular connection. The same program might act as a client on some connections and a server on others. The term "user agent" refers to any of the various client programs that initiate a request, including (but not limited to) browsers, spiders (web-based robots), command-line tools, custom applications, and mobile apps. The term "origin server" refers to the program that can originate authoritative responses for a given target resource. The terms "sender" and "recipient" refer to any implementation that sends or receives a given message, respectively.

HTTP relies upon the Uniform Resource Identifier (URI) standard [RFC3986] to indicate the target resource (Section 5.1) and relationships between resources. Messages are passed in a format similar to that used by Internet mail [RFC5322] and the Multipurpose Internet Mail Extensions (MIME) [RFC2045] (see Appendix A of [RFC7231] for the differences between HTTP and MIME messages).

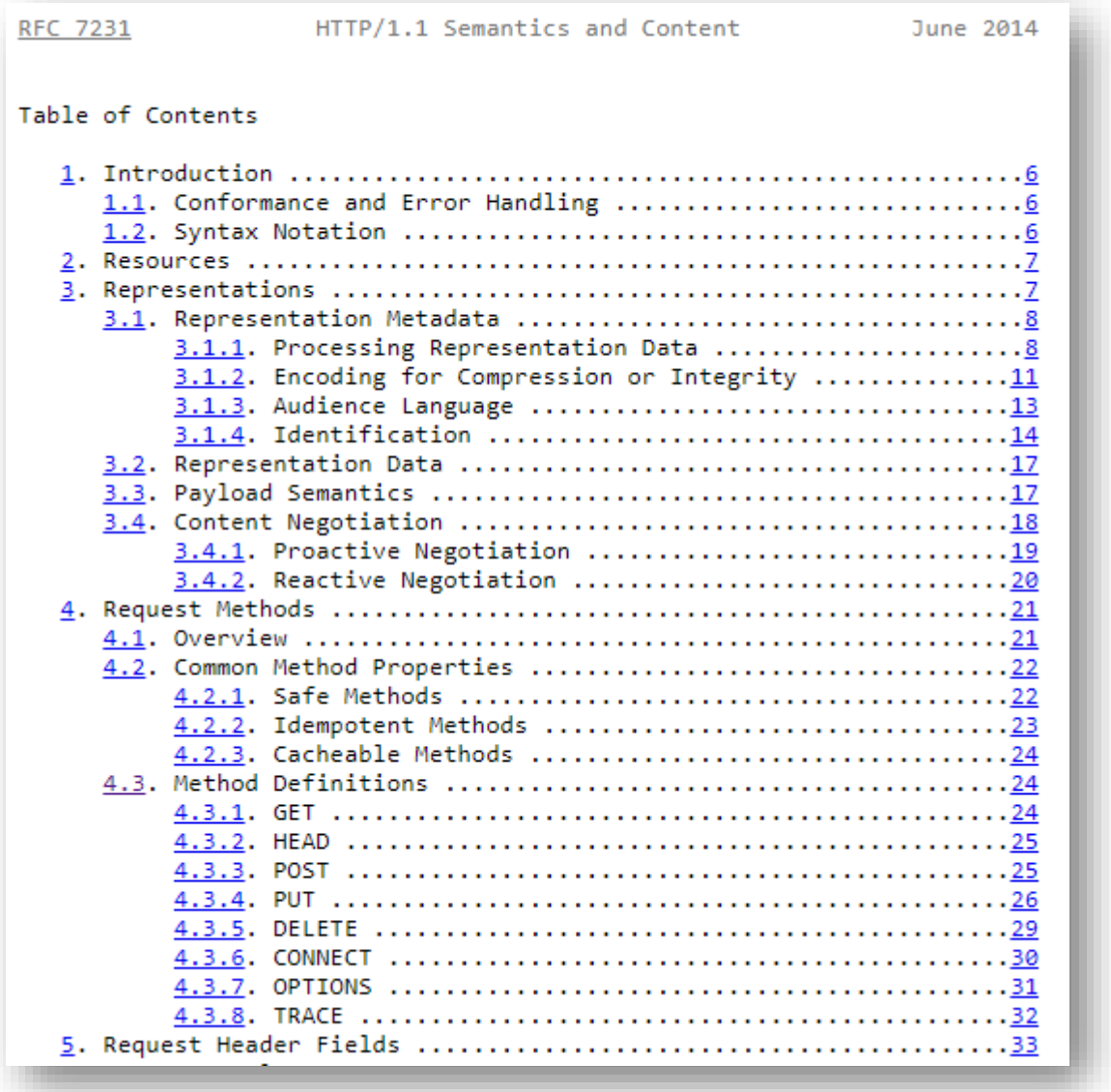
Most HTTP communication consists of a retrieval request (GET) for a representation of some resource identified by a URI. In the simplest case, this might be accomplished via a single bidirectional connection (===) between the user agent (UA) and the origin server (O).

```
         request   >
    UA ======================================= O
                                <   response
```

A client sends an HTTP request to a server in the form of a request message, beginning with a request-line that includes a method, URI, and protocol version (Section 3.1.1), followed by header fields containing request modifiers, client information, and representation metadata (Section 3.2), an empty line to indicate the end of the header section, and finally a message body containing the payload body (if any, Section 3.3).

2) HTTP

- **HTTP/1.1 (RFC7230, …, RFC7235)**
  - **Obsolete!**



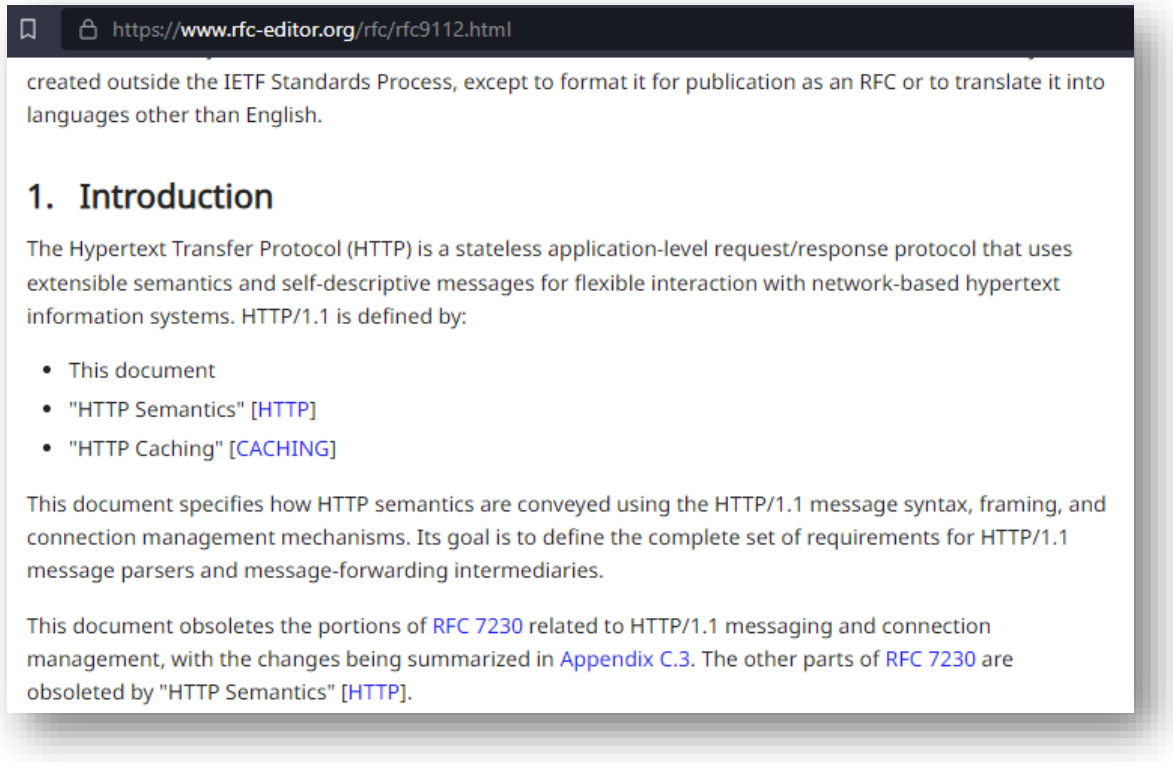
RFC 7231 HTTP/1.1 Semantics and Content June 2014

Table of Contents

1. Introduction 6
1.1. Conformance and Error Handling 6
1.2. Syntax Notation 6
2. Resources 7
3. Representations 7
3.1. Representation Metadata 8
3.1.1. Processing Representation Data 8
3.1.2. Encoding for Compression or Integrity 11
3.1.3. Audience Language 13
3.1.4. Identification 14
3.2. Representation Data 17
3.3. Payload Semantics 17
3.4. Content Negotiation 18
3.4.1. Proactive Negotiation 19
3.4.2. Reactive Negotiation 20
4. Request Methods 21
4.1. Overview 21
4.2. Common Method Properties 22
4.2.1. Safe Methods 22
4.2.2. Idempotent Methods 23
4.2.3. Cacheable Methods 24
4.3. Method Definitions 24
4.3.1. GET 24
4.3.2. HEAD 25
4.3.3. POST 25
4.3.4. PUT 26
4.3.5. DELETE 29
4.3.6. CONNECT 30
4.3.7. OPTIONS 31
4.3.8. TRACE 32
5. Request Header Fields 33

2) HTTP

- **HTTP/1.1 (RFC9110, RFC9111, RFC9112)**



More information: https://datatracker.ietf.org/doc/rfc9110/, https://www.rfc-editor.org/rfc/rfc9110.html

2) HTTP

- **HTTP/1.1 (RFC9110, RFC9111, RFC9112)**
- **HTTP/2 (RFC9113)**
- **HTTP/3 (RFC9114)**

9110 **HTTP Semantics** R. Fielding, M. Nottingham, J. Reschke [June 2022] (HTML, TEXT, PDF, XML) (Obsoletes RFC2818, RFC7230, RFC7231, RFC7232, RFC7233, RFC7235, RFC7538, RFC7615, RFC7694) (Updates RFC3864) (Also STD0097) (Status: INTERNET STANDARD) (Stream: IETF, Area: art, WG: httpbis) (DOI: 10.17487/RFC9110)

9111 **HTTP Caching** R. Fielding, M. Nottingham, J. Reschke [June 2022] (HTML, TEXT, PDF, XML) (Obsoletes RFC7234) (Also STD0098) (Status: INTERNET STANDARD) (Stream: IETF, Area: art, WG: httpbis) (DOI: 10.17487/RFC9111)

9112 **HTTP/1.1** R. Fielding, M. Nottingham, J. Reschke [June 2022] (HTML, TEXT, PDF, XML) (Obsoletes RFC7230) (Also STD0099) (Status: INTERNET STANDARD) (Stream: IETF, Area: art, WG: httpbis) (DOI: 10.17487/RFC9112)

9113 **HTTP/2** M. Thomson, C. Benfield [June 2022] (HTML, TEXT, PDF, XML) (Obsoletes RFC7540, RFC8740) (Status: PROPOSED STANDARD) (Stream: IETF, Area: art, WG: httpbis) (DOI: 10.17487/RFC9113)

9114 **HTTP/3** M. Bishop [June 2022] (HTML, TEXT, PDF, XML) (Status: PROPOSED STANDARD) (Stream: IETF, Area: tsv, WG: quic) (DOI: 10.17487/RFC9114)

2) HTTP

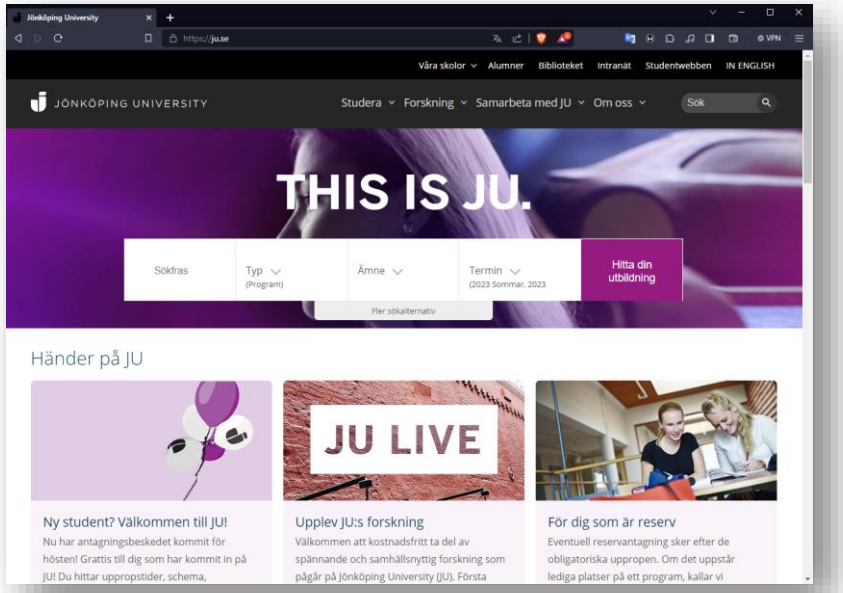
- The HTTP protocol is **STATELESS!**

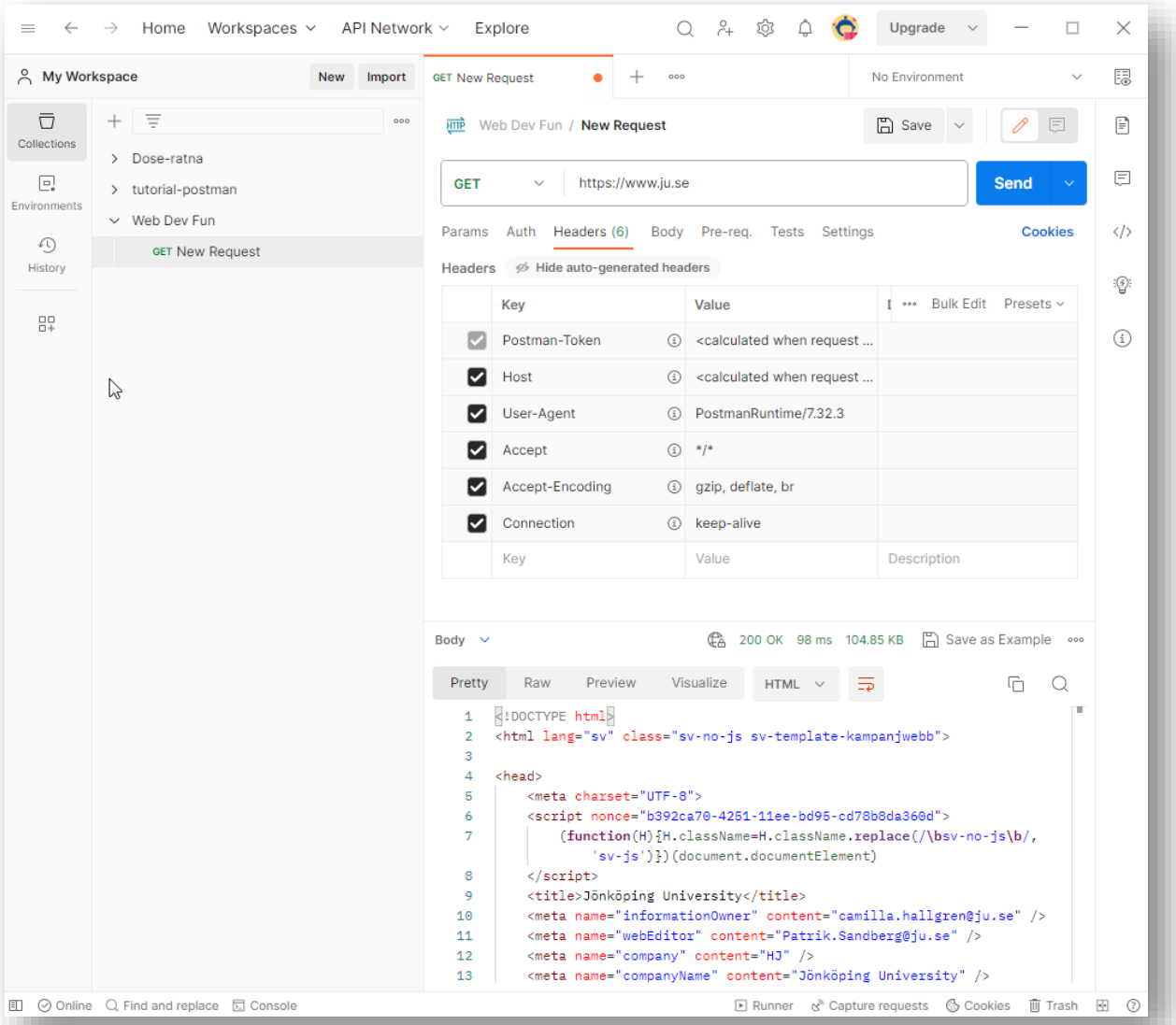
- It means that in http, there is **no mean to remember the state** of a web page, variable, value…
- Every **request** and **response** is **independant (**it knows **only itself)**

- If we want **memorization**, we will be forced to find another way:
  - **Cookies**
  - **Sessions**

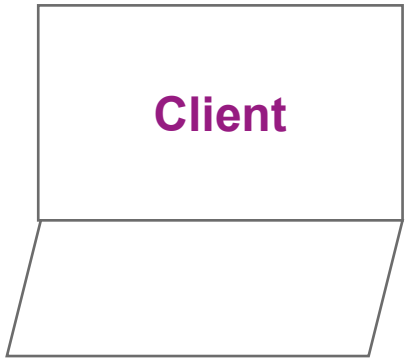
2) HTTP

- Example to access a web page: https://www.ju.se

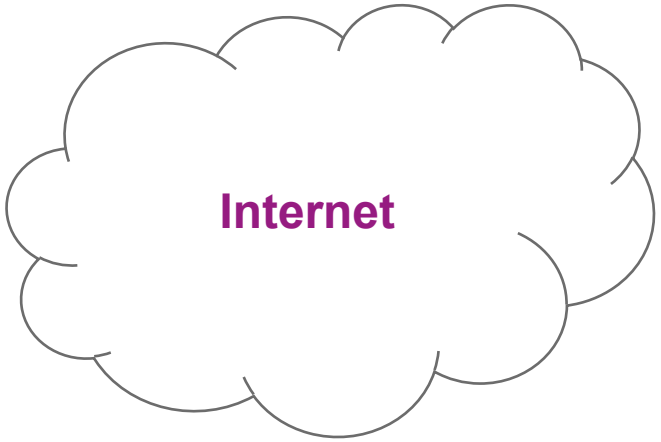


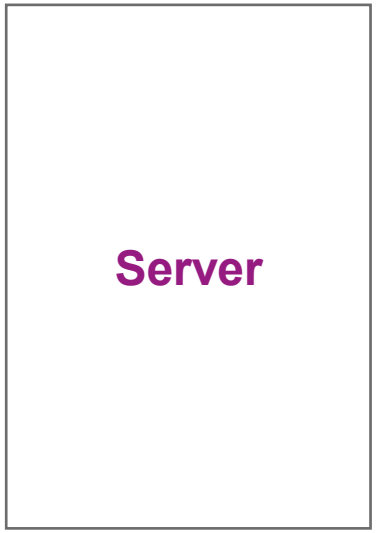


2) HTTP SESSION

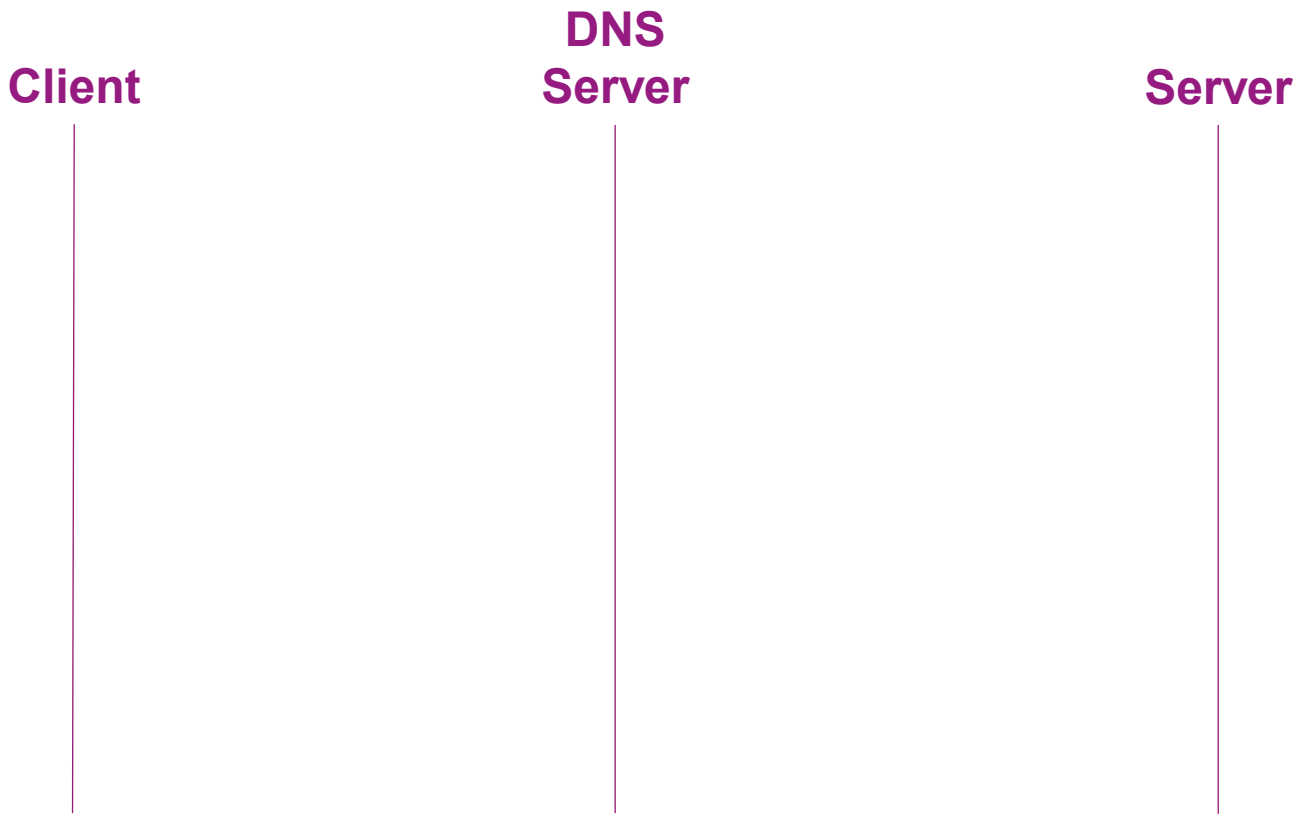


Browser
https://www.ju.se





2) HTTP SESSION

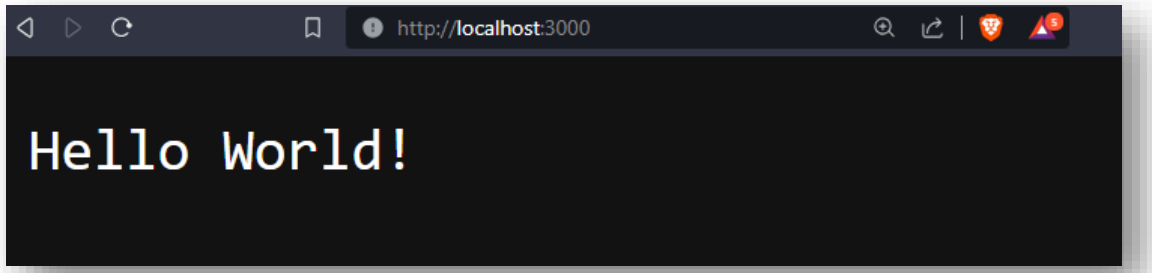


3) REQUEST & RESPONSE

hello-world.txt | server-04.js

mycv > hello-world.txt

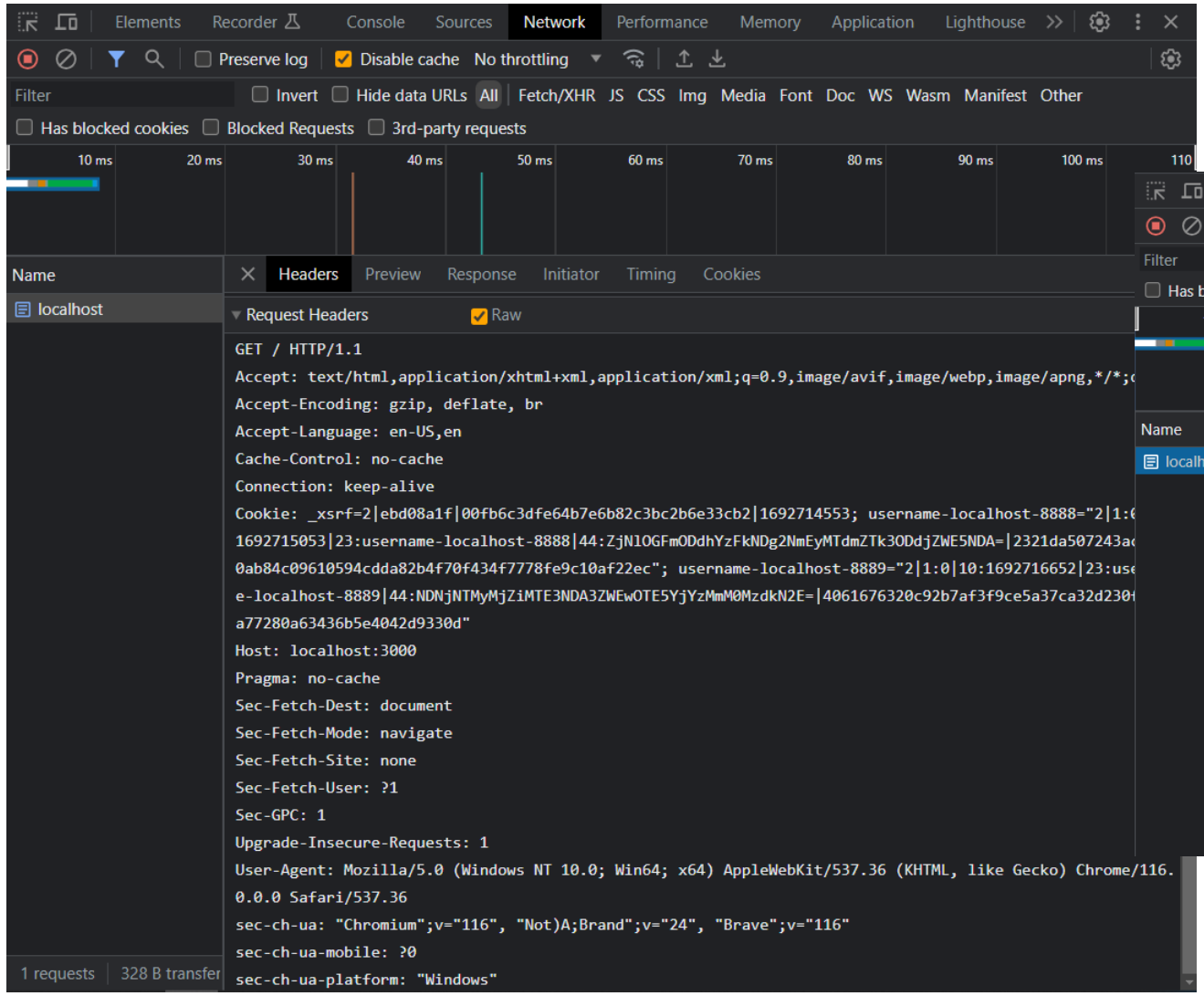
```
1  Hello World!
2
```



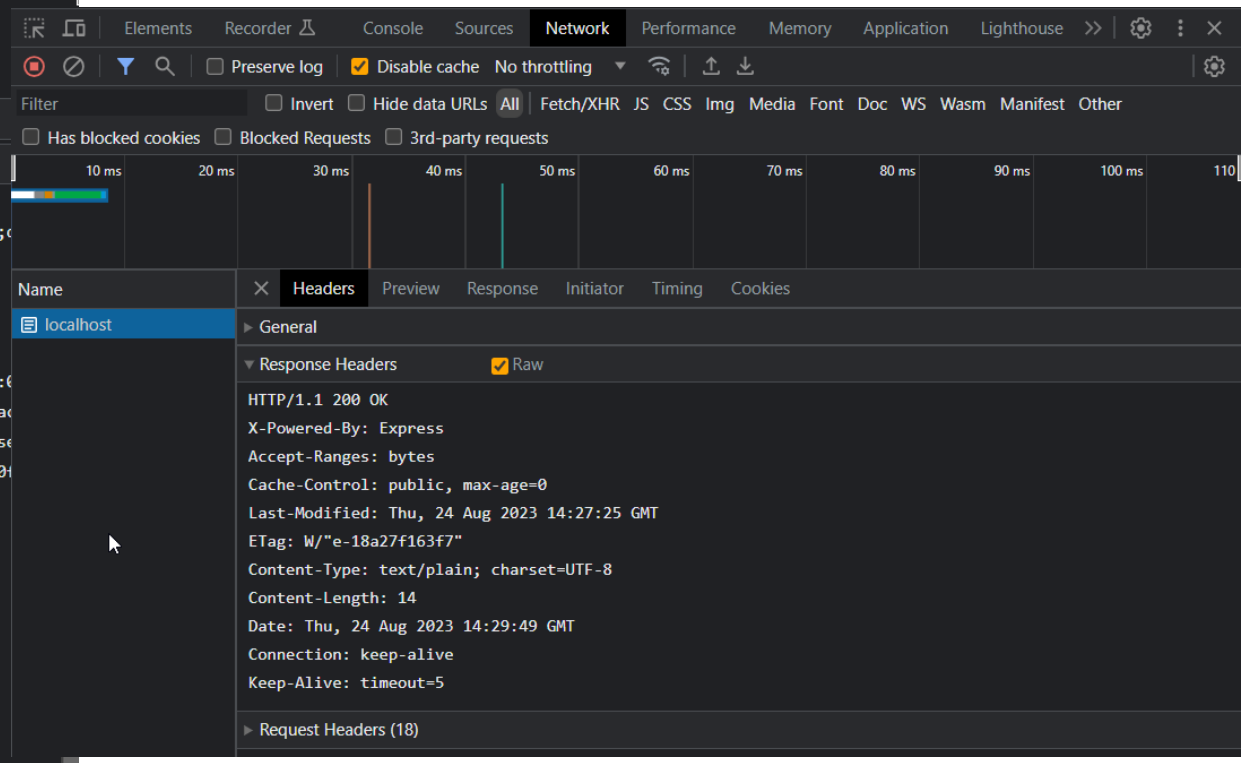
hello-world.txt | server-04.js

mycv > server-04.js > ...

```
1
2   const express = require('express');
3   const app = express();
4   const path = require('path');
5   const port = 3000;
6
7   // Without middleware
8   app.get('/', function (req, res, next) {
9       const options = {
10          root: path.join(__dirname)
11      };
12
13      res.sendFile('hello-world.txt', options, function (err) {
14          if (err) {
15              next(err);
16          } else {
17              console.log('File sent: OK');
18          }
19      });
20  });
21
22  app.listen(port, function (err) {
23      if (err) console.log(err);
24      console.log("Server listening on port: ", port, "...");
25      console.log("Working directory: ", __dirname)
26  });
27
```

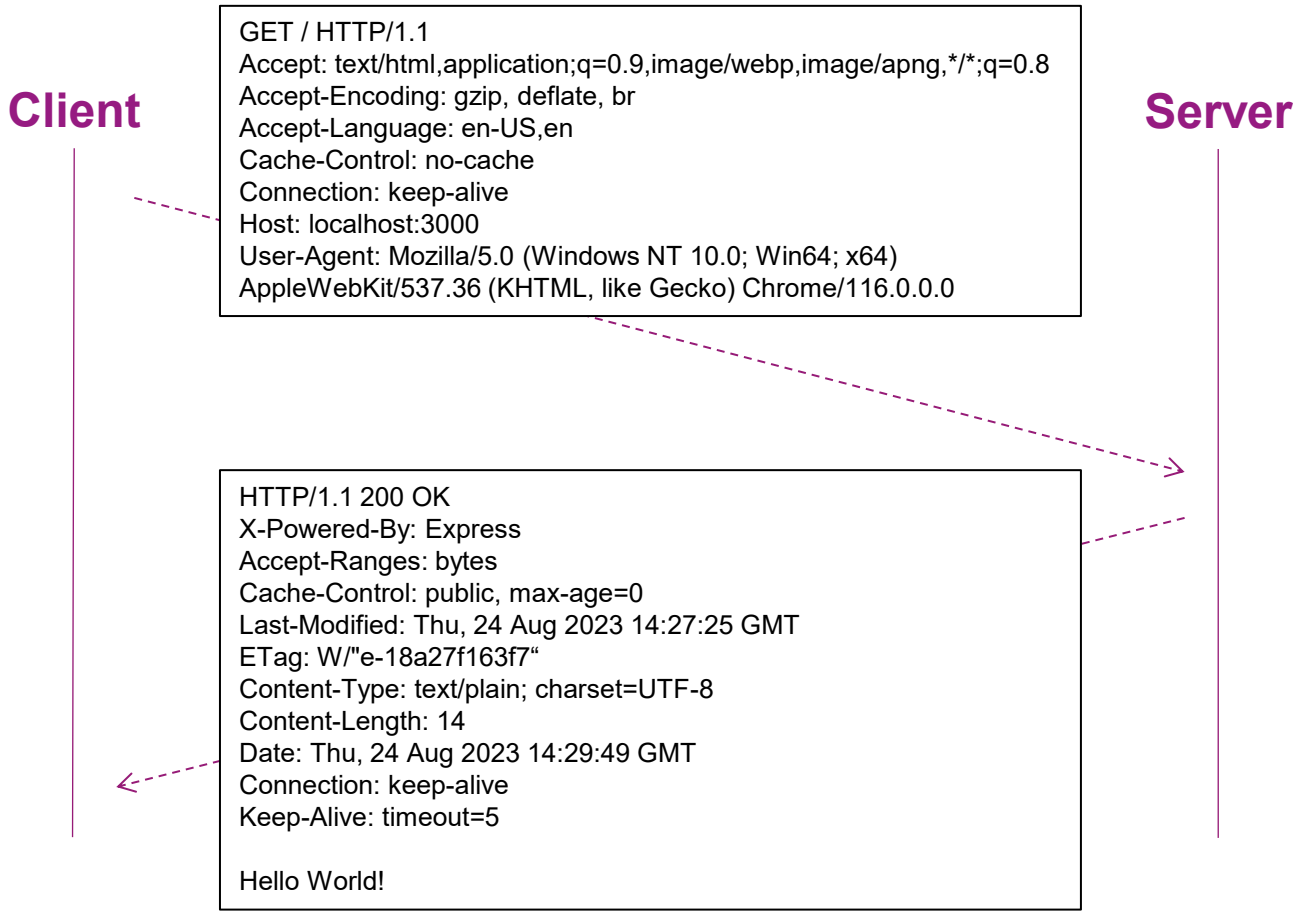


Request



Response

3) REQUEST & RESPONSE



hello-world.txt

Hello World!

- **Browsing the web** is basically **mainly** sending **GET http requests** and **getting http responses!**

3) REQUEST & RESPONSE

General

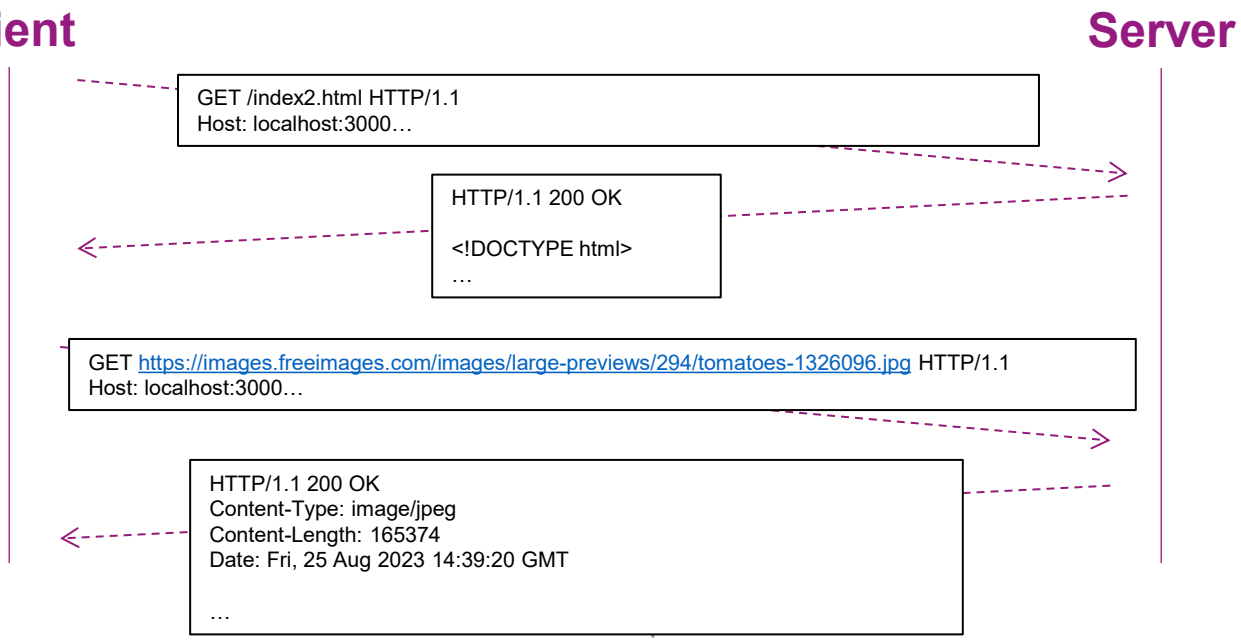
Request URL:	https://images.freeimages.com/images/large-previews/294/tomatoes-1326096.jpg
Request Method:	GET
Status Code:	200
Remote Address:	18.173.5.86:443
Referrer Policy:	strict-origin-when-cross-origin

Response Headers

Accept-Ranges:	bytes
Age:	888
Cache-Control:	max-age=31536000
Content-Length:	165374
Content-Type:	image/jpeg
Date:	Fri, 25 Aug 2023 12:55:33 GMT

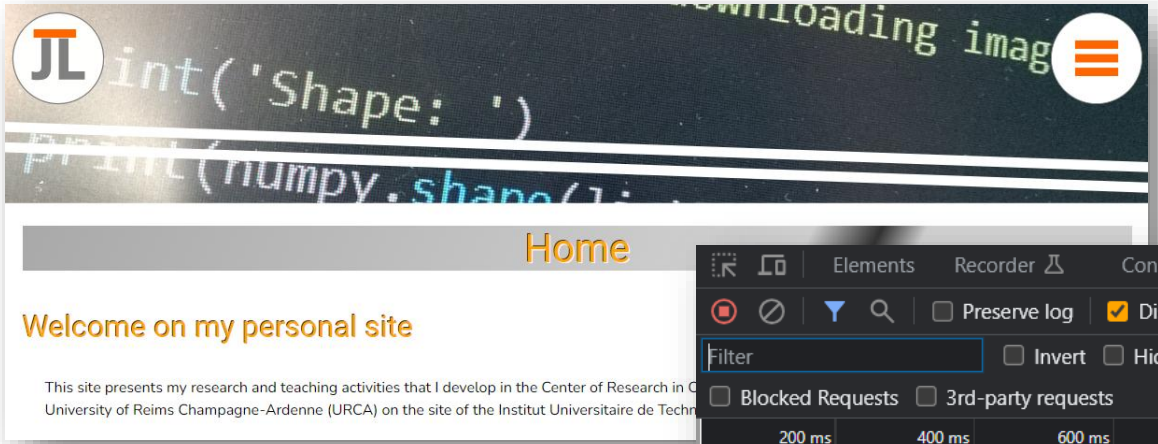
```html
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <p>I love tomatoes:</p>
        <img src="https://images.freeimages.com/images/large-previews/294/tomatoes-1326096.jpg" alt="Tomatoes" style="width:200px;" />
    </body>
</html>
```

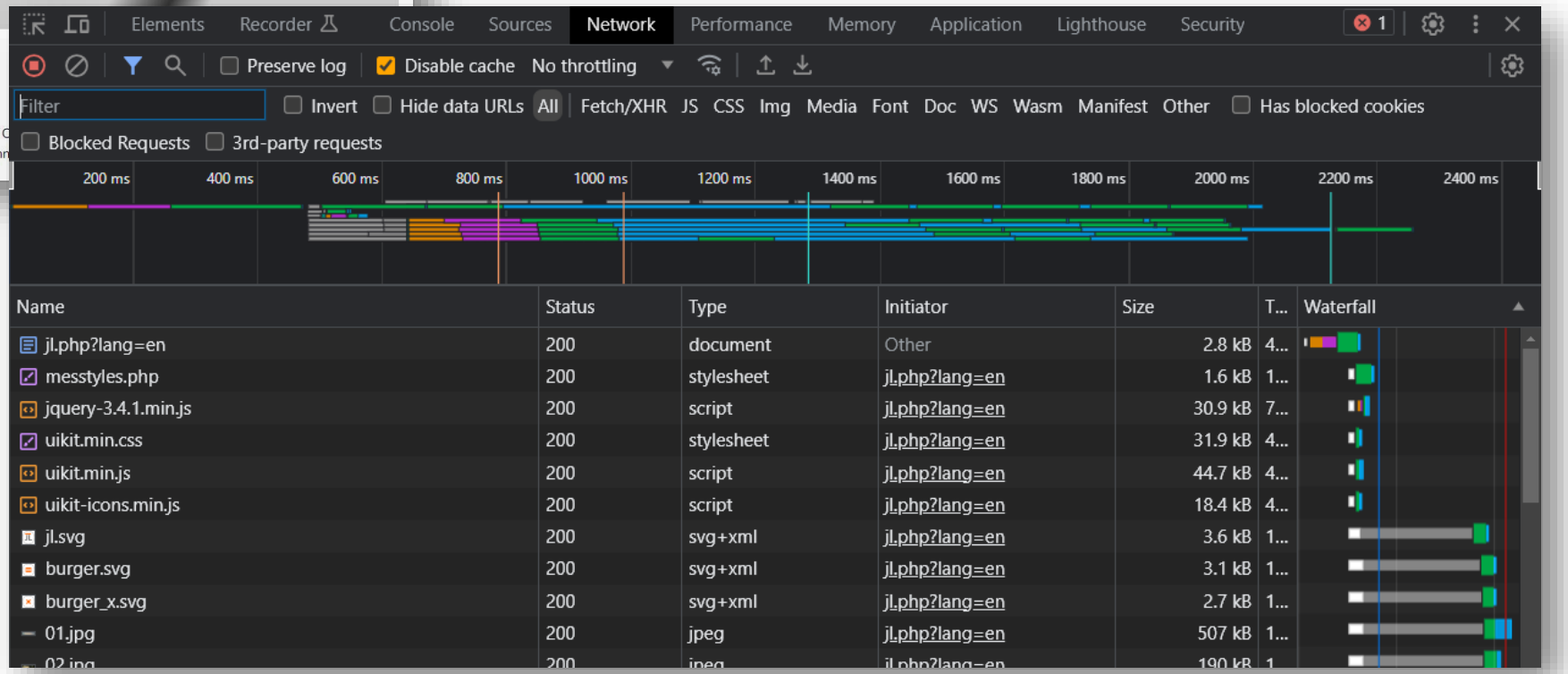
index2.html



- **Complicated page with many documents linked (CSS file, images, …):**

  https://jlandre.mmi-troyes.fr/jl.php?lang=en





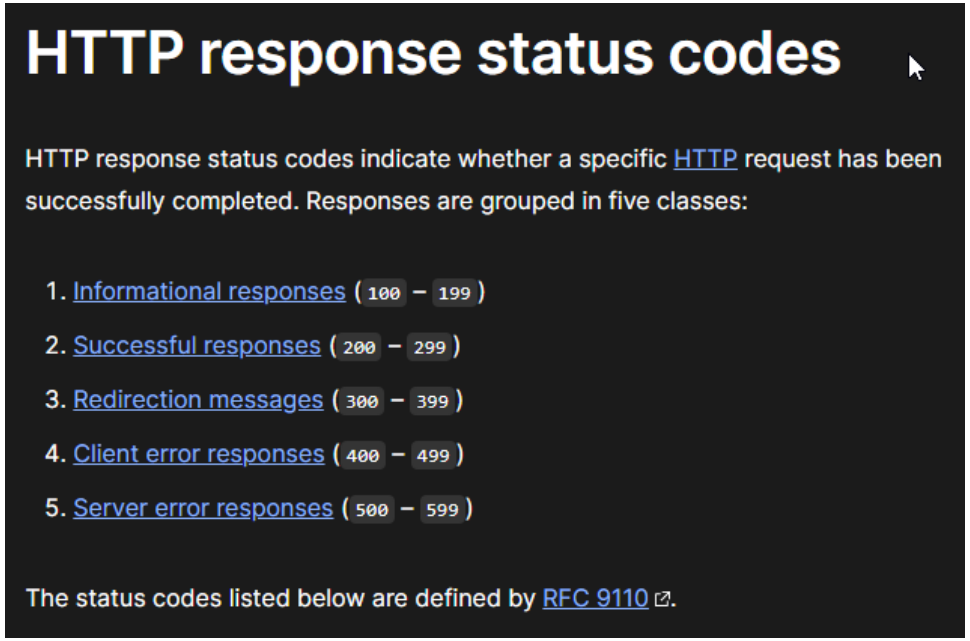
3) REQUEST & RESPONSE

- In case of **error**, the **server must inform the client** with a return **status code**

https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

https://www.digitalocean.com/community/tutorials/how-to-troubleshoot-common-http-error-codes

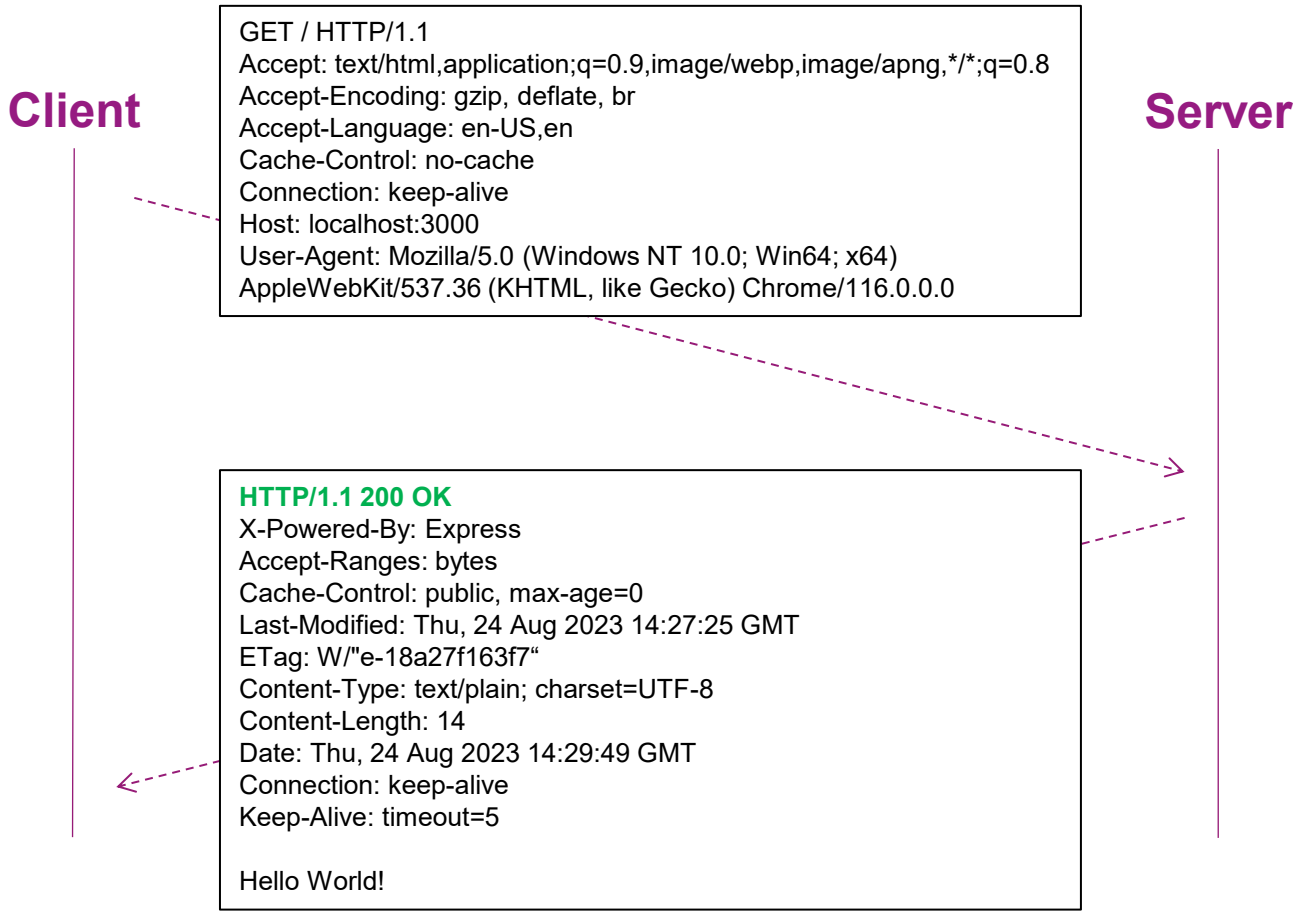
https://restfulapi.net/http-status-codes/



- 1xx: Informational
- 2xx: Success
- 3xx: Redirection
- 4xx: Client Error
- 5xx: Server Error

Images: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status, https://www.digitalocean.com/community/tutorials/how-to-troubleshoot-common-http-error-codes

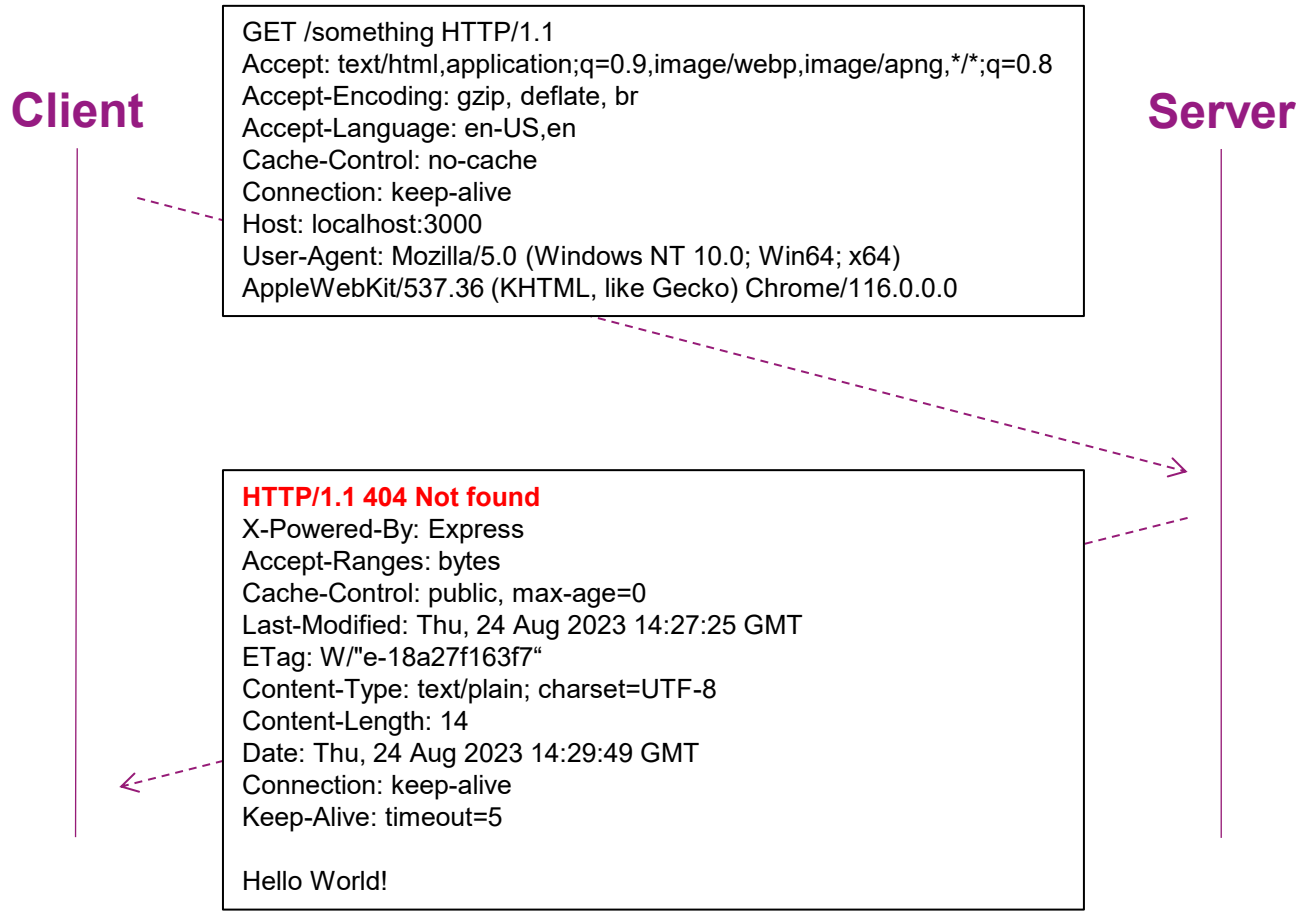
3) REQUEST & RESPONSE



hello-world.txt

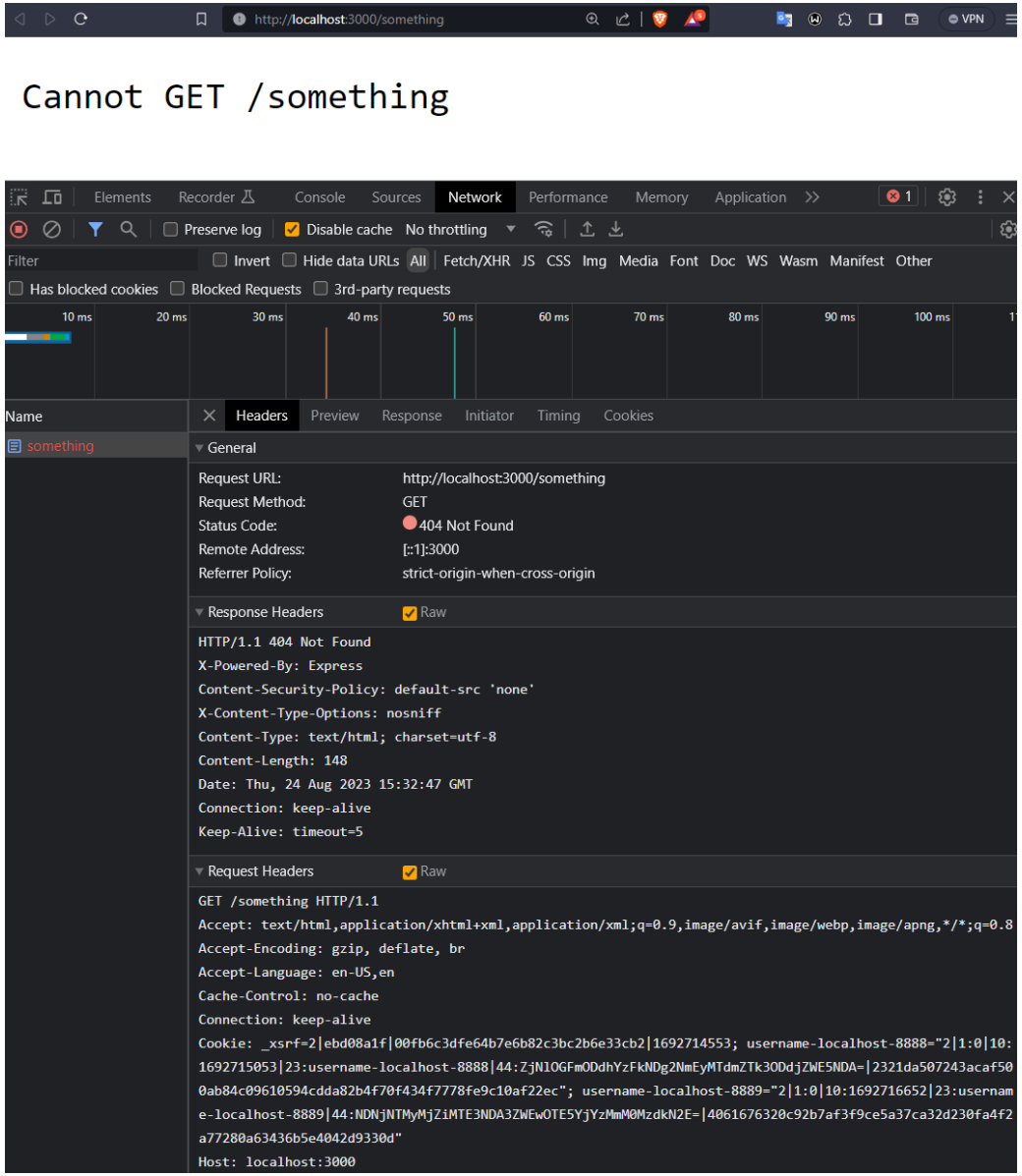
Hello World!

3) REQUEST & RESPONSE



hello-world.txt

Hello World!


http://localhost:3000/something
VPN
Cannot GET /something
Elements
Recorder
Console
Sources
Network
Performance
Memory
Application
Preserve log
Disable cache
No throttling
Filter
Invert
Hide data URLs
All
Fetch/XHR
JS
CSS
Img
Media
Font
Doc
WS
Wasm
Manifest
Other
Has blocked cookies
Blocked Requests
3rd-party requests
10 ms
20 ms
30 ms
40 ms
50 ms
60 ms
70 ms
80 ms
90 ms
100 ms
Name
something
Headers
Preview
Response
Initiator
Timing
Cookies
General
Request URL: http://localhost:3000/something
Request Method: GET
Status Code: 404 Not Found
Remote Address: [::1]:3000
Referrer Policy: strict-origin-when-cross-origin
Response Headers
Raw
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Security-Policy: default-src 'none'
X-Content-Type-Options: nosniff
Content-Type: text/html; charset=utf-8
Content-Length: 148
Date: Thu, 24 Aug 2023 15:32:47 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Request Headers
Raw
GET /something HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en
Cache-Control: no-cache
Connection: keep-alive
Cookie: _xsrf=2|ebd08a1f|00fb6c3dfe64b7e6b82c3bc2b6e33cb2|1692714553; username-localhost-8888="2|1:0|10:1692715053|23:username-localhost-8888|44:ZjN1OGFmODdhYzFkNDg2NmEyMTdmZTk3ODdjZWE5NDA=|2321da507243acaf500ab84c09610594cdda82b4f70f434f7778fe9c10af22ec"; username-localhost-8889="2|1:0|10:1692716652|23:username-localhost-8889|44:NDNjNTMyMjZiMTE3NDA3ZWEwOTE5YjYzMmM0MzdkN2E=|4061676320c92b7af3f9ce5a37ca32d230fa4f2a77280a63436b5e4042d9330d"
Host: localhost:3000

ANY QUESTIONS?